SCOUT AI INTERVIEWER

Project Files Reference Guide

Complete reference for every source file: classes, functions, dependencies, and data flow.

v1.0 • Internal Documentation

11	3	3,691+
Source Files	Entry Scripts	Questions

TOC Table of Contents

1 Project-Level Files

File	Purpose
.env	Stores secrets and runtime settings: OPENAI_API_KEY, QWEN_MODEL, OLLAMA_BASE_URL, EMBEDDING_MODEL, EMBEDDING_WORKERS, and path overrides. Never committed to version control.
requirements.txt	Python package dependencies: chromadb, openai, requests, openpyxl, python-dotenv.
questions_store.xlsx	Master question bank. Contains ~3,691 interview questions across roles, evaluation areas, difficulty levels, skill clusters, and scoring guidance. Raw data source for everything.
chroma_store/	Auto-created folder where ChromaDB persists its vector index on disk. Do not edit manually.
reports/	Auto-created folder where finished interview reports land (session_TIMESTAMP.json and report_TIMESTAMP.txt).
README.md	High-level architecture diagram and quick-start guide.

2 src/ — Core Library

2.1 config.py

What it does: Acts as the single source of truth for all runtime settings. It loads .env at import time and exposes every configurable value as class attributes on a Config class. A module-level singleton config is imported by every other file.

Key Contents

Symbol	Description
Config class	Holds all settings as class-level attributes (paths, model names, temperatures, ChromaDB settings, interview tuning knobs).
Config.PROJECT_ROOT	Absolute path to the repo root, used by all path-building code.
Config.EXCEL_FILE_PATH	Path to data/questions_store.xlsx.
Config.CHROMA_PERSIST_DIR	Path to chroma_store/.
Config.CHROMA_COLLECTION_NAME	Name of the ChromaDB collection ("scout_questions").
Config.QWEN_MODEL	Ollama model used for interviewing (qwen2.5:7b by default).
Config.EMBEDDING_MODEL	Separate model used only for generating embeddings (nomic-embed-text).
Config.EMBEDDING_WORKERS	Number of parallel threads used when embedding the question bank (default: 8).
Config.OPENAI_EVAL_MODEL	OpenAI model used for meta-evaluation (gpt-4o-mini).
Config.TOP_K	Number of RAG candidates to retrieve per question slot (default: 8).
Config.MAX_HISTORY	Rolling conversation window size fed to Qwen (default: 5 turns).
Config.COLUMNS	A dict mapping column names to 0-based column indices in the Excel sheet. Used by excel_reader.py.
config (singleton)	Module-level instance; imported as: from .config import config

Depends on: `python-dotenv`, `pathlib`, `os`

Used by: Every other file in src/ and scripts/.

2.2 `embedder.py`

What it does: Converts text strings into float vector embeddings by calling the nomic-embed-text model running locally in Ollama. Supports parallel embedding (using ThreadPoolExecutor) to dramatically speed up bulk operations like loading the full question bank.

Key Contents

Symbol	Description
Embedder class	Main embedding class. Initialises with model name, Ollama base URL, and worker count from config.
Embedder.encode(texts, show_progress)	Embeds a list of texts. Automatically uses parallel execution for large batches and falls back to sequential for tiny ones. Returns a list of float vectors in the same order as inputs.
Embedder.encode_one(text)	Embeds a single string with no threading overhead. Used during live RAG retrieval.
Embedder.check_connection()	Hits Ollama's /api/tags endpoint to verify the embedding model is available. Raises a clear error if Ollama is not running.
Embedder._embed_single(text)	The atomic unit — sends one POST to Ollama's /api/embeddings with retry logic (up to 3 attempts, exponential backoff).
_MAX_RETRIES = 3	Module-level constant for retry limit.

Key Design Decisions

- Uses nomic-embed-text (274 MB, dedicated embedding model) instead of Qwen to get ~10-20x speed improvement during data loading.
- Each thread creates its own HTTP request (thread-safe because requests.post is stateless).
- Results dict is indexed by position to preserve original order after async completion.

Depends on: `config.py`, `requests`, `concurrent.futures`

Used by: `vector_store.py` (bulk ingestion and query embedding).

2.3 excel_reader.py

What it does: Reads questions_store.xlsx using openpyxl in read-only mode and converts each row into a typed Question dataclass. Handles missing values gracefully and supports both streaming (memory-efficient) and full-load modes.

Key Contents

Symbol	Description
Question dataclass	Represents one row from the Excel sheet. Has 23 fields covering question text, role, evaluation area, experience level, scoring guidance (what_ai_listens_for, strong_signal_example, weak_signal_example), and metadata.
Question.to_document()	Builds the text corpus string that gets embedded and stored in ChromaDB. Combines question text, role, evaluation area, skills, type, and scoring guidance into a single searchable document.
Question.to_metadata()	Returns a flat dict of all fields in ChromaDB-safe types (str | int | float | bool). Stored alongside the embedding and retrieved at query time.
ExcelQuestionReader class	Opens the Excel file and provides iteration/load methods.
ExcelQuestionReader.iter_questions(active_only)	Generator that streams questions one by one. Skips rows with empty question_id or question_text, and optionally skips inactive questions.
ExcelQuestionReader.load_all(active_only)	Loads all questions into a Python list. Convenience wrapper over iter_questions.
ExcelQuestionReader.get_summary()	Returns a dict with counts broken down by role, question type, and experience level. Used by load_data.py --summary.

Depends on: `config.py`, `openpyxl`

Used by: `scripts/load_data.py` (to ingest data into ChromaDB).

2.4 vector_store.py

What it does: Wraps a ChromaDB PersistentClient to provide a clean interface for storing and querying question embeddings. All operations go through this file — writing (upsert), semantic search, metadata filtering, and ID lookup.

Key Contents

Symbol	Description
VectorStore class	Main persistence layer. Opens or creates the ChromaDB collection on init.
VectorStore.upsert(ids, documents, embeddings, metadatas)	Inserts or updates a batch of pre-embedded documents. Idempotent — safe to re-run.
VectorStore.ingest_questions(questions, batch_size)	High-level ingestion: takes a list of Question objects, generates embeddings in batches via Embedder.encode(), and upserts them. This is the main Excel → ChromaDB pipeline.
VectorStore.search(query_text, top_k, where)	Core RAG retrieval method. Embeds the query with Embedder.encode_one(), then performs cosine similarity search in ChromaDB. Returns results as dicts with id, document, distance, similarity, and all metadata fields.
VectorStore.filter_by_metadata(where, limit)	Pure metadata filter with no semantic scoring. Used for exact-match lookups (e.g., all questions for a specific role).
VectorStore.get_by_id(question_id)	Fetch one question by its exact ID. Used by inspect_db.py.
VectorStore.nuke_and_recreate()	Drops and recreates the collection. Used by load_data.py --force-reload.
VectorStore.count (property)	Returns the number of documents in the collection.
VectorStore.is_populated (property)	Returns True if the collection has at least one document.
VectorStore._parse_results(raw)	Static helper that flattens ChromaDB's nested response format into a clean list of dicts. Also computes similarity = 1 - distance for cosine space.

Depends on: `config.py`, `embedder.py`, `chromadb`

Used by: `rag_engine.py` (queries), `scripts/load_data.py` (ingestion), `scripts/inspect_db.py` (inspection).

2.5 `rag_engine.py`

What it does: Provides a business-language query API on top of VectorStore. Instead of raw ChromaDB calls, callers use methods like search_for_parameter(evaluation_area, role, ...). Also builds the correct metadata filter $and / single condition structures that ChromaDB expects.

Key Contents

Symbol	Description
RAGEngine class	High-level wrapper around VectorStore.
RAGEngine.search_for_parameter(evaluation_area, role, experience_level, question_type, top_k)	Primary method used during an interview. Constructs a natural language query from the evaluation area and role, applies role/level filters, and returns ranked candidates.
RAGEngine.search_by_role(role, query, top_k, ...)	Semantic search scoped to a role with a custom query string.
RAGEngine.get_follow_ups(parent_id, limit)	Retrieves follow-up questions linked to a parent question via metadata filter on parent_question_id.
RAGEngine.get_by_metadata(role, evaluation_area, ...)	Pure metadata filter with no semantic scoring.
RAGEngine.random_sample(role, experience_level, n, pool_size)	Returns a random sample of n questions from a filtered pool. Used for variety.
RAGEngine._build_filter(role, experience_level, question_type)	Static helper: builds a ChromaDB $and / single-field filter dict from optional parameters.

Depends on: `config.py`, `vector_store.py`

Used by: `interview_runner.py` (RAG retrieval during each interview turn).

2.6 qwen_interviewer.py

What it does: Contains the full Qwen-7B interviewing logic. Qwen has two roles: (1) choosing which question to ask from RAG candidates and (2) scoring the candidate's answer. Both are done via structured JSON prompts sent to Ollama's chat API.

Key Contents

Symbol	Description
QwenClient class	Thin REST client for Ollama's /api/chat endpoint. Handles retry logic (3 attempts, exponential backoff) for connection errors and timeouts.
QwenClient.chat(messages, temperature)	Sends a list of {role, content} messages and returns Qwen's response string.
QwenClient.prompt(user_text, system, temperature)	Single-turn convenience wrapper that builds the messages list and calls chat().
QwenClient.check_connection()	Verifies Ollama is running and the Qwen model is available.
QwenInterviewer class	Higher-level class that uses QwenClient to perform the two core interviewing tasks.
QwenInterviewer.select_question(...)	Feeds RAG candidates to Qwen with a structured prompt asking it to pick the best question (0-based index) and whether to rephrase it. Returns a dict with question_text, question_id, rephrased, reason, and rag_context. Falls back to the top RAG result if JSON cannot be parsed.
QwenInterviewer.score_response(...)	Sends a detailed scoring prompt to Qwen with the candidate's answer, evaluation criteria, and signal examples. Returns raw_score (0-10), signal_strength, rationale, strengths, weaknesses, needs_follow_up. Uses temperature=0.3 for consistent scoring.

Symbol	Description
QwenInterviewer._parse_selection(raw, candidates)	Extracts JSON from Qwen's selection response (handles markdown code blocks).
QwenInterviewer._parse_score(raw)	Extracts and clamps the scoring JSON.
QwenInterviewer._format_candidates(candidates)	Formats the RAG candidate list as a numbered text block for the Qwen prompt.
QwenInterviewer._extract_rag_context(question_data)	Pulls evaluation guidance fields from ChromaDB metadata for use in the scoring prompt.

Depends on: `config.py`, `requests`, `json`, `re`

Used by: `interview_runner.py` (steps 2 and 4 of each interview turn).

2.7 openai_evaluator.py

What it does: The second-layer AI in the system. After Qwen interviews and scores, OpenAI's gpt-4o-mini audits Qwen's work — it checks question appropriateness and whether Qwen's score was fair and consistent. OpenAI does NOT conduct the interview.

Key Contents

Symbol	Description
QuestionQualityResult dataclass	OpenAI's verdict on a single question: relevance_score, difficulty_score, clarity_score, overall_quality (all 0-10), verdict (approved / acceptable / poor), and free-text feedback.
ScoringAccuracyResult dataclass	OpenAI's verdict on Qwen's scoring: qwen_score, openai_score (OpenAI's own independent score), score_delta (OpenAI - Qwen), verdict (accurate / over-scored / under-scored / inconsistent), is_accurate (True when |delta| <= 1.5), and qualitative assessment.

Symbol	Description
MetaEvaluationResult dataclass	Container combining both results for one interview turn. Also holds alerts (list of critical issues) and has_quality_issue / has_scoring_issue flags.
MetaEvaluationResult.print_diagnostics()	Prints a formatted terminal block with icons (checkmark / warning / cross) showing both verdicts for the current turn.
OpenAIMetaEvaluator class	Orchestrates the two-part audit. Uses openai.OpenAI client.
OpenAIMetaEvaluator.evaluate_turn(...)	Public method called once per interview turn. Runs both _evaluate_question_quality() and _evaluate_scoring_accuracy(), assembles alerts, and returns a MetaEvaluationResult.
OpenAIMetaEvaluator._evaluate_question_quality(...)	Sends a structured prompt to OpenAI asking it to rate the question on relevance, difficulty, and clarity. Parses the JSON response into a QuestionQualityResult.
OpenAIMetaEvaluator._evaluate_scoring_accuracy(...)	Sends a second structured prompt with the full context and asks OpenAI for its own independent score. Returns ScoringAccuracyResult.
OpenAIMetaEvaluator._call_openai(prompt)	Raw OpenAI API call with retry logic for rate limits (RateLimitError) and server errors.
OpenAIMetaEvaluator._parse_json(raw)	Extracts JSON from OpenAI's response (handles markdown code fences).

Alert Thresholds: Question verdict of "poor" → generates a POOR QUESTION alert. |score_delta| > 2 → generates a SIGNIFICANT SCORING ERROR alert.

Depends on: `config.py`, `openai`, `json`, `re`

Used by: `interview_runner.py` (step 5 of each turn), `interview_state.py` (to store meta-eval results).

2.8 interview_state.py

What it does: Owns all mutable session state during an interview. Tracks every Q&A turn, live scores per evaluation area, the rolling conversation window fed to Qwen, the dedupe set of asked question IDs, and OpenAI's audit results. Nothing is persisted to disk here — that happens in report_generator.py.

Key Contents

Symbol	Description
ConversationTurn dataclass	One complete Q&A exchange. Holds the question, candidate's response, evaluation area, Qwen's score/rationale/strengths/weaknesses, and OpenAI's audit verdict (added later via update_meta_eval()).
EvaluationAreaTracker dataclass	Live score tracker for one evaluation area. Tracks questions_asked, running average avg_qwen_score, weighted_score, and whether the min_score threshold has been breached.
EvaluationAreaTracker.update(qwen_score)	Called after each turn for the relevant area. Recalculates averages and checks threshold.
EvaluationAreaTracker.is_complete (property)	Returns True when questions_asked >= questions_to_ask.
InterviewSession dataclass	The complete session state object. Has the rolling conversation_history (last 5 turns, Qwen-only), the untruncated full_turn_log (all turns, used for reports), area_tracker dict, overall score, and meta-eval alert list.
InterviewSession.to_dict() / to_json()	Serialises the session to a JSON-safe dict/string for export.
InterviewStateManager class	Manages the session lifecycle.
InterviewStateManager.start_session(role, experience_level, areas)	Creates a new InterviewSession and sets up EvaluationAreaTracker for each configured area.

Symbol	Description
InterviewStateManager.record_turn(...)	Called after each Q&A. Creates a ConversationTurn, appends to both the rolling window and full log, adds question ID to the dedupe set, and calls EvaluationAreaTracker.update().
InterviewStateManager.update_meta_eval(turn_number, ...)	Attaches OpenAI audit results to the matching turn in full_turn_log. Also increments qwen_question_issues / qwen_scoring_issues counters. Does NOT update the rolling conversation_history — Qwen must not see its own audit.
InterviewStateManager.is_complete()	Returns True when all areas have their required number of questions.
InterviewStateManager.get_next_area()	Returns the name of the next area needing questions, prioritised by highest weight first.
InterviewStateManager.get_context_string()	Formats the last 5 turns for injection into Qwen's system prompt. Only includes Qwen's own Q+score data — no candidate responses, no OpenAI content.
InterviewStateManager.get_live_scorecard()	Returns a formatted scorecard string showing per-area and overall scores. Printed at the end of the interview.
InterviewStateManager.save_to_file(filepath)	Serialises the session to JSON and writes to disk.

Depends on: `json`, `datetime`, `uuid`

Used by: `interview_runner.py` (every turn), `report_generator.py` (to read session data for reports).

2.9 interview_runner.py

What it does: The central orchestrator. Wires together all components (ChromaDB, RAGEngine, QwenInterviewer, OpenAIMetaEvaluator, InterviewStateManager, CandidateSimulator) and drives the interview loop turn by turn.

Key Contents

Symbol	Description
InterviewRunner class	Main orchestrator class. Holds references to all component instances.
InterviewRunner.setup_session(...)	Initialises all components in sequence: (1) open ChromaDB, (2) connect Qwen, (3) optionally connect OpenAI meta-evaluator and/or candidate simulator. Then calls InterviewStateManager.start_session().
InterviewRunner.run_interactive()	Runs the interview in interactive CLI mode (human types answers). Loops until is_complete(), calling _run_turn() for each area. Handles turn failures with a consecutive-failure counter — after 3 failures, area is force-marked complete.
InterviewRunner.run_simulated()	Same loop as run_interactive() but delegates candidate answer generation to CandidateSimulator. Fully automated, no human input.
InterviewRunner.run_demo(candidate_responses)	Demo mode — accepts a pre-written list of answers to inject, useful for testing.
InterviewRunner._run_turn(area, turn_num, mode, demo_response)	Core turn logic — executes one complete interview turn in 7 steps: 1) RAG retrieval, 2) Qwen selects question, 3) get candidate response, 4) Qwen scores response, 5) record turn in state manager, 6) optional OpenAI meta-evaluation, 7) print progress.
InterviewRunner.get_session()	Returns the InterviewSession object for post-interview report generation.
InterviewRunner.save_session(filepath)	Delegates to InterviewStateManager.save_to_file().

RAG Fallback Chain in _run_turn()

Step 1: search by area + role + level (strict)

Step 2: search by area + role (no level filter)

Step 3: search by area + role with TOP_K x 4 (larger pool, no level filter)

Step 4: if all found IDs were already asked → expand to TOP_K x 6 to find unasked questions

Step 5: if still no unasked → raise ValueError (turn fails, runner retries up to 3x)

Depends on: config.py, vector_store.py, rag_engine.py, embedder.py, qwen_interviewer.py, openai_evaluator.py, interview_state.py, candidate_simulator.py

Used by: `scripts/run_interview.py`

2.10 report_generator.py

What it does: Reads a completed InterviewSession and generates three forms of output: a terminal summary, a structured JSON file, and a human-readable recruiter text report.

Key Contents

Symbol	Description
ReportGenerator class	Stateless — takes a session object and formats it.
ReportGenerator.print_summary(session)	Prints a condensed terminal scorecard: per-area Qwen scores with signal labels (STRONG / MODERATE / WEAK), breach warnings, overall score, and a meta-evaluation summary showing any critical alerts.
ReportGenerator.save_json(session, filepath)	Serialises the full session to JSON (session.to_json()) and writes it to disk. Creates parent directories if needed.
ReportGenerator.save_text(session, filepath)	Builds and saves a multi-section recruiter report as a .txt file.
ReportGenerator._build_text_report(session)	Assembles the text report in 4 sections: (1) overall score, (2) per-area breakdown, (3) OpenAI meta-evaluation summary and alerts, (4) full interview transcript with Qwen and OpenAI audit details per turn.

Report Sections (text file)

- SECTION 1: OVERALL SCORE
- SECTION 2: EVALUATION AREA BREAKDOWN
- SECTION 3: OPENAI META-EVALUATION OF QWEN
- SECTION 4: INTERVIEW TRANSCRIPT

Depends on: `interview_state.py`, `json`, `datetime`, `pathlib`

Used by: `scripts/run_interview.py` (after the interview finishes).

2.11 candidate_simulator.py

What it does: Uses OpenAI GPT to generate realistic candidate responses automatically, enabling fully automated pipeline testing without a human. Three built-in personas define the quality and style of generated responses.

Key Contents

Symbol	Description
PERSONAS dict	Three pre-defined persona descriptions: "strong" (8 years B2B SaaS, STAR answers, metrics), "average" (3 years, adequate but vague), "weak" (< 1 year, generic, rambling).
CandidateSimulator class	Wraps an openai.OpenAI client with a chosen persona.
CandidateSimulator.__init__(persona)	Accepts "strong", "average", "weak", or a custom description string. Raises ValueError if OPENAI_API_KEY is not set.
CandidateSimulator.answer(question, role, experience_level, evaluation_area, conversation_context)	Builds a system prompt from the persona and a user prompt from the question context. Calls OpenAI at temperature=0.85 for natural variation. Returns a 3-6 sentence candidate response. Retries up to 3 times on failure.
CandidateSimulator.persona_name (property)	Returns the persona label string.

Note: Used only in --simulate mode. Does not participate in normal interactive interviews.

Depends on: `config.py`, `openai`

Used by: `interview_runner.py` (when simulate=True).

3 scripts/ — Entry Points & Utilities

3.1 run_interview.py

What it does: The main CLI entry point for running an interview. Parses all command-line arguments, builds the area configuration, sets up the InterviewRunner, runs the interview, and saves reports.

CLI Arguments

Argument	Description
--role TEXT	Job role (default: "Sales Executive").
--level	Experience level: Fresher, Early Career, Mid-level, Senior (default: Mid-level).
--areas AREA ...	One or more area strings in the format "AREA_NAME:WEIGHT:QUESTIONS:MIN_SCORE:NON_NEGOTIABLE". If omitted, uses the default 5-area Sales configuration.
--no-meta-eval	Skips OpenAI meta-evaluation (faster, no API cost).
--simulate	Runs in fully automated mode — OpenAI generates candidate answers.
--persona	Candidate persona for simulation: strong, average, weak (default: average).
--export DIR	Directory to save reports (default: reports/).
--no-export	Skips saving reports.
--verbose	Enables INFO-level logging.

Key Functions

Function	Description
parse_area_string(area_str)	Parses a "AREA:WEIGHT:QUESTIONS:MIN_SCORE:NN" string into a dict.
print_config(role, level, areas, ...)	Prints a formatted configuration table before the interview starts.
main()	Argument parsing → config print → InterviewRunner.setup_session() → run_interactive() → ReportGenerator.print_summary() → save JSON + text reports.

Default Area Configuration (when --areas is not given)

Area	Weight	Questions	Min Score	Non-Negotiable
Objection Handling	25%	1	6.0	Yes
Communication Skills	20%	1	—	No
Pipeline Management	20%	1	—	No
Client Relationship	20%	1	—	No
Resilience & Grit	15%	1	—	No

3.2 `load_data.py`

What it does: One-time (or periodic) data pipeline that reads questions_store.xlsx, embeds all questions using nomic-embed-text, and writes them into ChromaDB. Supports incremental sync so that re-embedding the full dataset is only needed when questions are actually added or removed.

CLI Modes

Flag	Behaviour
(no flag)	Full load — embeds all active questions. Use on first run.
--sync	Incremental sync — computes diff between Excel IDs and ChromaDB IDs. Only embeds new rows; deletes removed rows; skips unchanged rows entirely.
--force-reload	Wipes ChromaDB (nuke_and_recreate()) then runs a full load.
--delete	Wipes ChromaDB and exits without reloading. Prompts for confirmation.
--summary	Prints Excel statistics (counts by role, type, level) without touching ChromaDB.
--workers N	Overrides EMBEDDING_WORKERS for this run.

Key Functions

Function	Description
main()	Orchestrates the 4-step pipeline: read Excel → verify Ollama → open ChromaDB → sync/load.
_run_incremental_sync(store, excel_questions)	Computes the diff (new IDs, removed IDs, unchanged IDs) and applies only the necessary changes.
_get_all_chroma_ids(store)	Fetches all document IDs from ChromaDB in batches of 1,000 (IDs only, no embeddings — very fast).
_delete_from_store(store, ids)	Deletes a list of question IDs from ChromaDB.
_print_summary(reader)	Prints Excel statistics table.
_delete_collection()	Handles the --delete flow with a confirmation prompt.

Run Order: This script must be run BEFORE run_interview.py. It is a one-time setup step.

3.3 `inspect_db.py`

What it does: A diagnostic utility for inspecting what is currently stored in ChromaDB. Useful for verifying a successful load, testing semantic search, or checking a specific question by ID.

CLI Modes

Flag	Behaviour
(no flag)	Shows collection stats (name, document count) and a sample of 5 active questions.
--search QUERY	Performs a semantic search with optional --role filter.
--role ROLE [--level LEVEL]	Pure metadata filter — lists questions for a given role/level.
--id QUESTION_ID	Fetches and displays all metadata for a single question by its ID.
--top-k N	Number of results to show (default: 5).

Key Functions

Function	Description
main()	Argument parsing → opens store → branches on search/role/id/default.
_print_results(results)	Formats and prints a list of ChromaDB result dicts.

4 Data Flow Summary

Ingestion Pipeline

```
questions_store.xlsx
        |
        v  (scripts/load_data.py)
ExcelQuestionReader  ──reads──>  List[Question]
        |
        v  (src/embedder.py — nomic-embed-text via Ollama, parallel)
Embedder.encode()  ──generates──>  List[List[float]]  (embeddings)
        |
        v  (src/vector_store.py)
VectorStore.ingest_questions()  ──upserts──>  ChromaDB (chroma_store/)
```

Interview Pipeline

```
scripts/run_interview.py
        |
        v
InterviewRunner.setup_session()
        |
        └── [per turn] InterviewRunner._run_turn(area)
                |
                ├─ Step 1: RAGEngine.search_for_parameter()
                |           └── VectorStore.search()
                |               └── Embedder.encode_one() → ChromaDB
cosine search
                |               └── returns List[QueryResult] (candidates)
                |
                ├─ Step 2: QwenInterviewer.select_question(candidates)
                |           └── Qwen-7B picks best question + optionally
rephrases
                |
                ├─ Step 3: Get candidate response
                |           (interactive input /
CandidateSimulator.answer())
                |
                ├─ Step 4: QwenInterviewer.score_response()
                |           └── Qwen-7B scores 0-10 with rationale
                |
                ├─ Step 5: InterviewStateManager.record_turn()
```

```
                  |           └── Updates rolling context, area trackers,
dedupe set
                  |
                  └ Step 6: OpenAIMetaEvaluator.evaluate_turn()
                              └── Audits Qwen question quality + scoring
accuracy
                              └── InterviewStateManager.update_meta_eval()
```

```
         |
         v (after all turns)
ReportGenerator.print_summary()
ReportGenerator.save_json()   ──>  reports/session_TIMESTAMP.json
ReportGenerator.save_text()   ──>  reports/report_TIMESTAMP.txt
```

5 Model Responsibilities at a Glance

Model	Where	What it does
`nomic-embed-text (Ollama)`	embedder.py	Generates vector embeddings ONLY — for question bank indexing and query embedding during RAG retrieval. Never generates text.
`qwen2.5:7b (Ollama)`	qwen_interviewer.py	THE INTERVIEWER. Selects questions from RAG candidates, optionally rephrases them, and scores candidate responses 0-10 with rationale.
`gpt-4o-mini (OpenAI)`	openai_evaluator.py	THE META-EVALUATOR. Audits Qwen's question quality and scoring accuracy after each turn. Does NOT conduct the interview.
`gpt-4o-mini (OpenAI)`	candidate_simulator.py	THE SIMULATED CANDIDATE. Generates realistic candidate responses in --simulate mode only. Not used in live interactive interviews.